pfi

Python for Fixed Income

COURSE 6 · NEURAL NETWORKS

Momentum, Adam, and Learning Rate Schedules

WEEK 2 · DAY 2

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The optimizer is the rule for the step

The loop inside every fit

1. Compute the loss on a batch
2. Work out each weight's gradient
3. **Move each weight: the optimizer**
4. Repeat

Four rules for step 3

SGD	learning rate x gradient
Momentum	keeps part of the last step
Adam	gradient / its running size
RMSprop	Adam without m or correction

First on day 1's neuron, checked against Keras; then on the 64-32 network, chosen on the stopping rows.

Plain SGD, as Keras writes it

```
self.assign_sub(variable, ops.multiply(gradient, learning_rate))   # keras/src/optimizers/sgd.py
```

0.01

SGD's default learning rate

0

SGD's default momentum

One step, learning rate 1	Intercept	Weight
Keras SGD, by apply	-1.058284	0.533916
netkit.gradient_step	-1.058284	0.533916

optimizer.apply(gradients, variables): the optimizer on its own, as fit uses it.

Equal within 1e-6: Keras's plain step is day 1's gradient_step.

How momentum works

1. Last step

kept as a velocity

2. Add the gradient

velocity = m x velocity - rate x g

3. Move

the weights move by the velocity

4. Overshoot

a long rate with 0.9 overshoots

You control

The momentum and the learning rate, which interact.

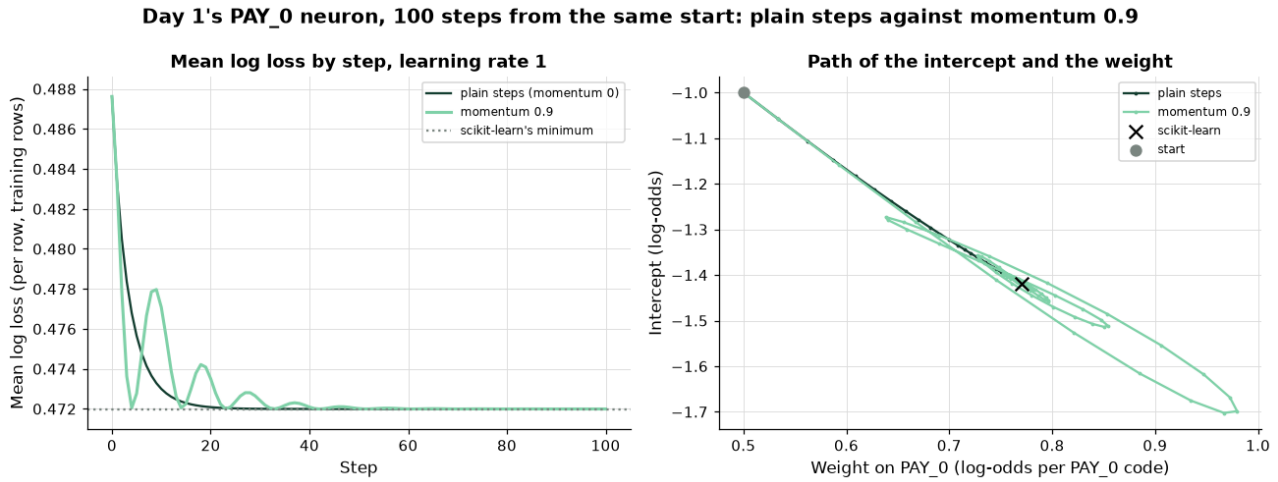
What comes out

The same update, with a velocity carried along.

Momentum by hand: it carries the last step

```
velocity = momentum * velocity - lr * gradient
weight   = weight + velocity
```

Two steps	Intercept	Weight
momentum_step	-1.159484	0.593217
Keras SGD(0.9)	-1.159484	0.593217



Learning rate 1: plain steps settle from step 20, momentum 0.9 from step 48. Momentum overshoots.

How Adam works

1. m

running average of the gradient

2. v

running average of its square

3. The step

alpha x m / sqrt(v)

4. Correct

a bias correction at the start

You control

The learning rate, then beta_1, beta_2, and epsilon.

What comes out

SGD's update, with two numbers per weight.

Adam by hand, and the bias correction

```
m = m + (g - m) * (1 - beta_1);   v = v + (g**2 - v) * (1 - beta_2)
alpha = lr * sqrt(1 - beta_2**t) / (1 - beta_1**t)
w = w - alpha * m / (sqrt(v) + epsilon)
```

First step moves	Intercept	Weight
Plain step, rate 1	0.058284	0.033916
Adam, rate 0.05	0.049997	0.049995
Adam without 1 - beta_1^t	0.005000	0.005000
Adam with no correction	0.158105	0.158099

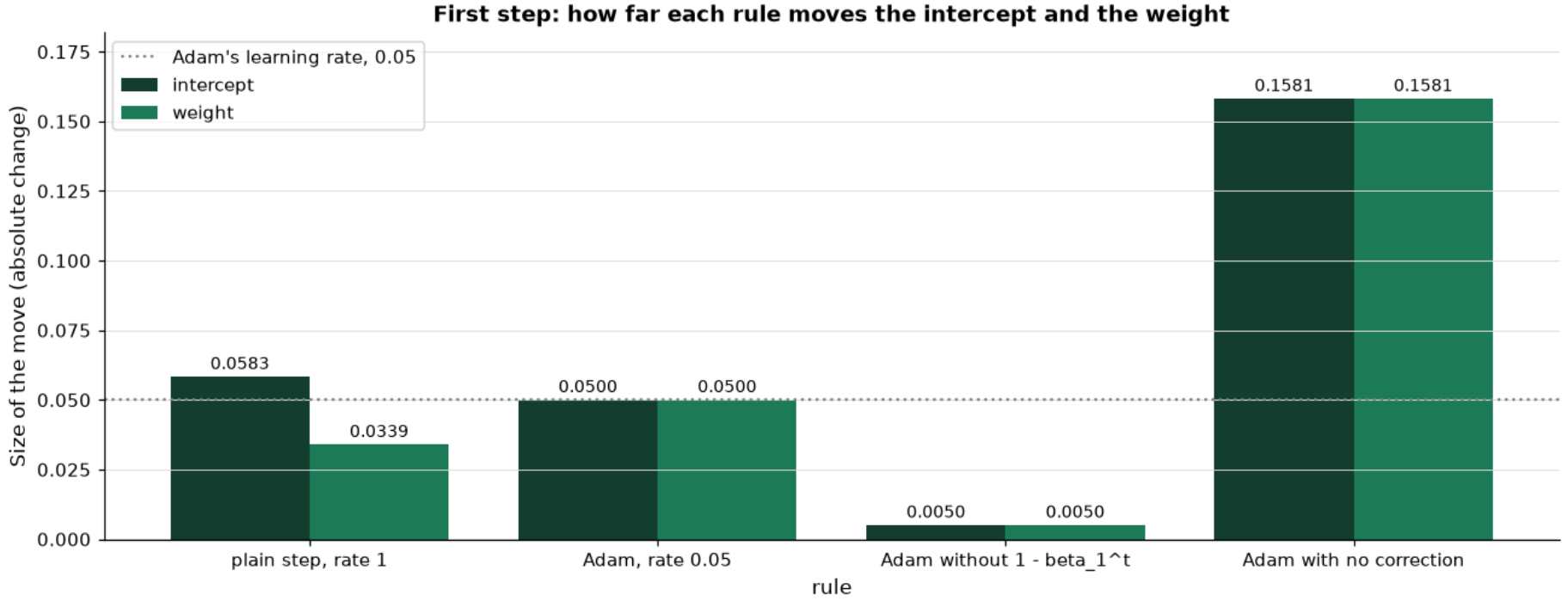
Alpha at steps 1, 2, 3

`0.015811, 0.011766, 0.010101`

The correction fades toward the learning rate. Three steps equal Keras's Adam within 1e-6.

Adam moves every weight by the learning rate at first, whatever its gradient.

The first step, drawn



Plain step: two heights. Adam: level pairs, 0.049995 each; a tenth without 1 - beta_1^t; 0.158099 with no correction.

Where the name Adam comes from

Diederik P. Kingma and Jimmy Ba, 22 December 2014

"Adam: A Method for Stochastic Optimization"

arXiv 1412.6980; published at the 3rd International Conference for Learning Representations, 2015

keras.optimizers.Adam carries its name from this title.

Source, read 6 October 2026: the arXiv record of the paper, arxiv.org/abs/1412.6980. Bibliographic facts and the record's comment only; the paper itself was not read for this course.

How RMSprop works

1. v only

average of the squared gradient

2. Divide

gradient / sqrt(v)

3. Same size

every weight moves by about the rate

4. Wobble

steps do not shrink near the minimum

You control

The learning rate, rho, momentum, and centered.

What comes out

An Adam-like step without the m.

RMSprop, and the defaults Keras reads out

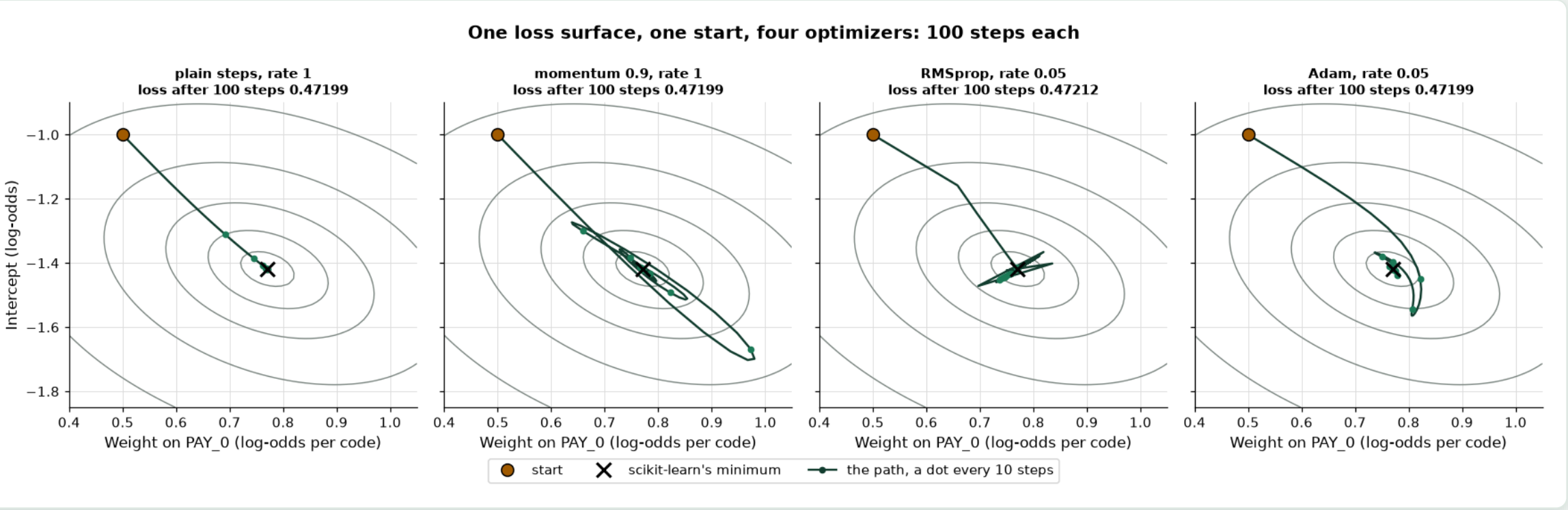
Optimizer	learning_rate	momentum	rho	beta_1	beta_2	epsilon
SGD	0.01	0	-	-	-	-
RMSprop	0.001	0	0.9	-	-	1e-07
Adam	0.001	-	-	0.9	0.999	1e-07

```
v = rho * v + (1 - rho) * g**2
w = w - lr * g / sqrt(v + epsilon)        # epsilon inside the square root
```

One step at 0.05 on the neuron: both weights move 0.1580, about 3.16 times the rate.

From get_config: SGD defaults to 0.01, RMSprop and Adam to 0.001.

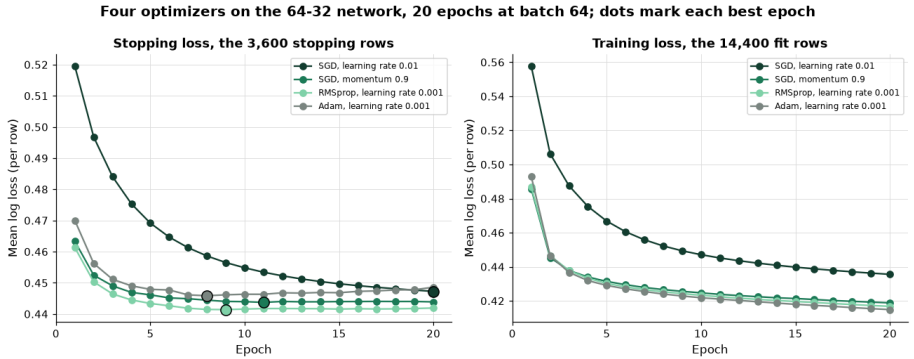
Four optimizers, one loss surface



Plain steps walk straight in; momentum swings past the cross; Adam curves in; RMSprop wobbles round it. On a two-weight bowl, plain steps are as good as any.

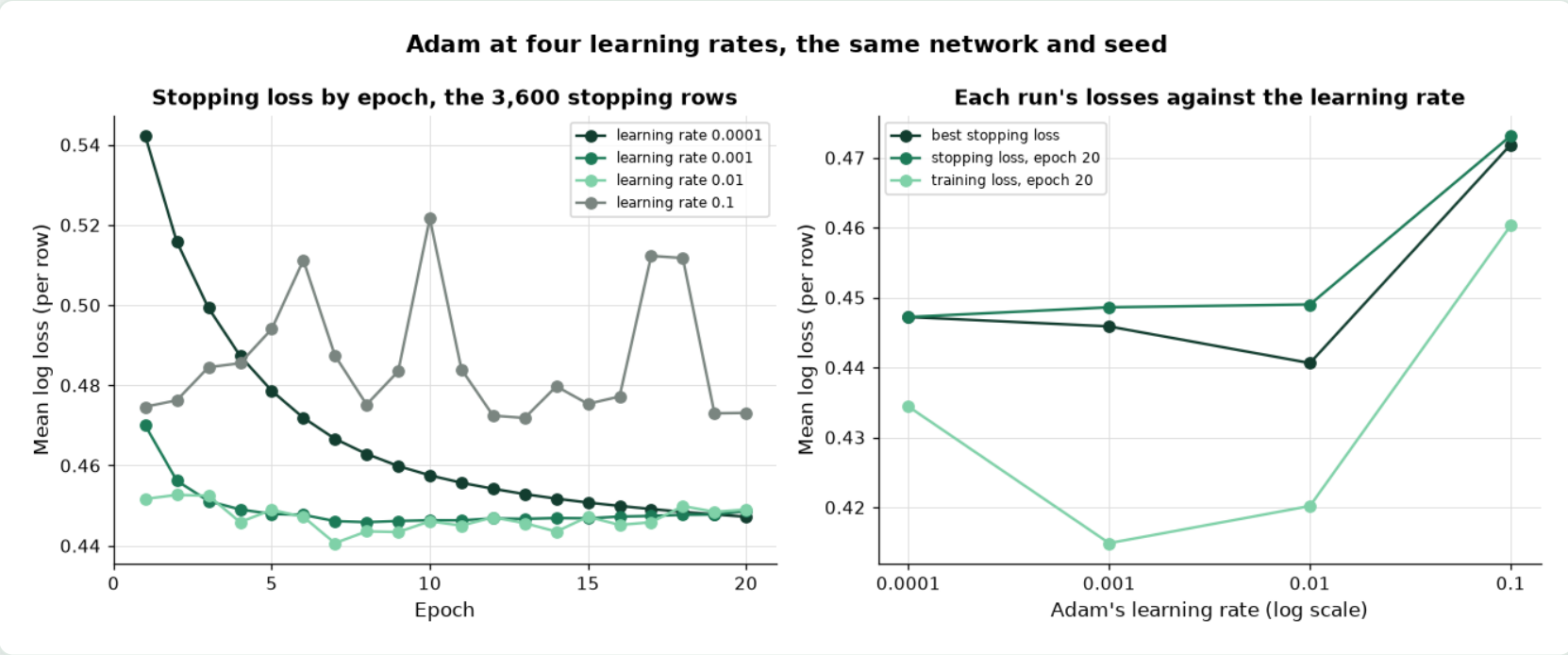
Four optimizers on one network, 20 epochs

Optimizer	Best epoch	Best stopping loss	Training loss, epoch 20
SGD, learning rate 0.01	20	0.4473	0.4356
SGD, momentum 0.9	11	0.4437	0.4188
RMSprop, learning rate 0.001	9	0.4414	0.4168
Adam, learning rate 0.001	8	0.4459	0.4149



The lowest training loss is not the lowest stopping loss. The stopping rows choose.

Adam's learning rate: too small, too large



Rate	Best	Epoch
0.0001	0.4472	20
0.001	0.4459	8
0.01	0.4406	7
0.1	0.4719	13

0.01 is lowest but ragged: largest rise between epochs 0.0040.

Too small is slow; too large never arrives (day 1's neuron at learning rate 20 did the same).

Every setting of Adam and RMSprop

Setting	Default	What it controls	Tune first
Adam: learning_rate	0.001	step size of every weight	tune first
Adam: beta_1	0.9	memory of the gradient average m	
Adam: beta_2	0.999	memory of the squared-gradient average v	
Adam: epsilon	1e-07	keeps the division by sqrt(v) safe	
Adam: amsgrad	False	v keeps the largest value it has seen	
RMSprop: learning_rate	0.001	step size, scaled by recent gradient size	tune first
RMSprop: rho	0.9	memory of v	
RMSprop: momentum	0.0	velocity on top of the step	
RMSprop: epsilon	1e-07	added to v inside the square root	
RMSprop: centered	False	divide by a variance estimate	

The other ten settings are SGD's, with the same defaults. The learning rate is the one to tune first; epsilon is the one with a visible effect when it is far from its default.

How a schedule works

1. Rate by step

a function of the step count

2. Start large

get near a fit fast

3. Then shrink

settle, no bouncing

4. Pass it in

as the learning rate of the optimizer

You control

The shape, the starting rate, how fast it falls.

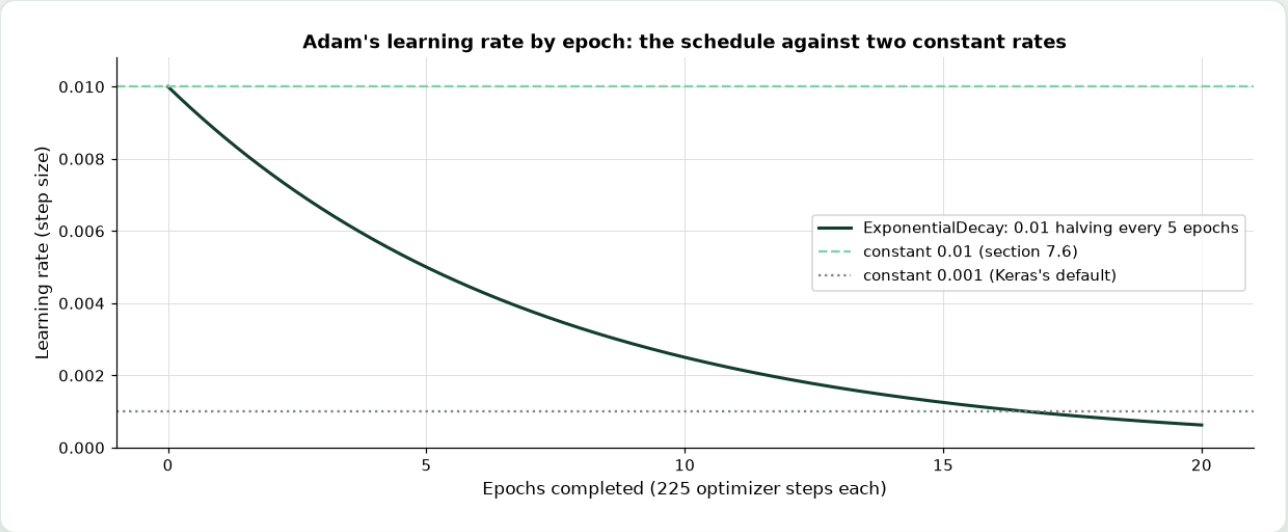
What comes out

A rate that changes at every step.

A schedule: start large, then halve

```
lr(t) = 0.01 * 0.5 ** (t / 1125)
```

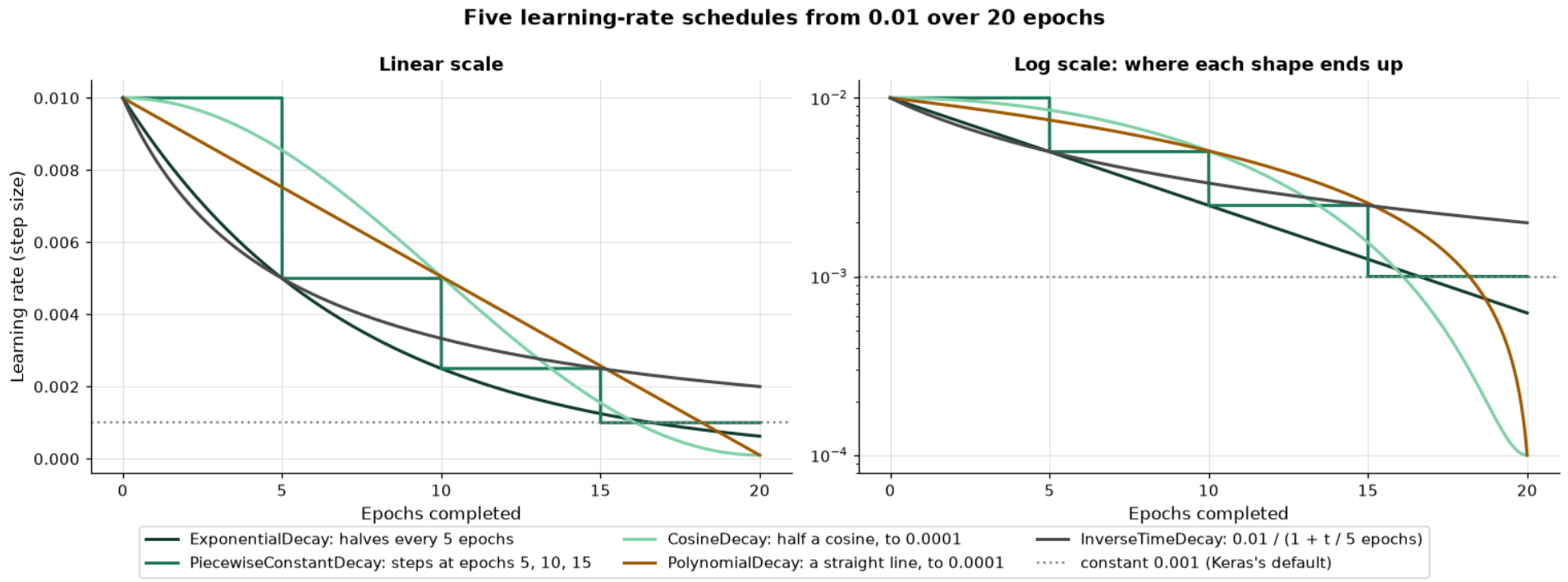
Epoch	Step	Learning rate
1	0	0.010000
6	1,125	0.005000
11	2,250	0.002500
16	3,375	0.001250
20	4,275	0.000718



4,500 steps; learning rate now 0.000625

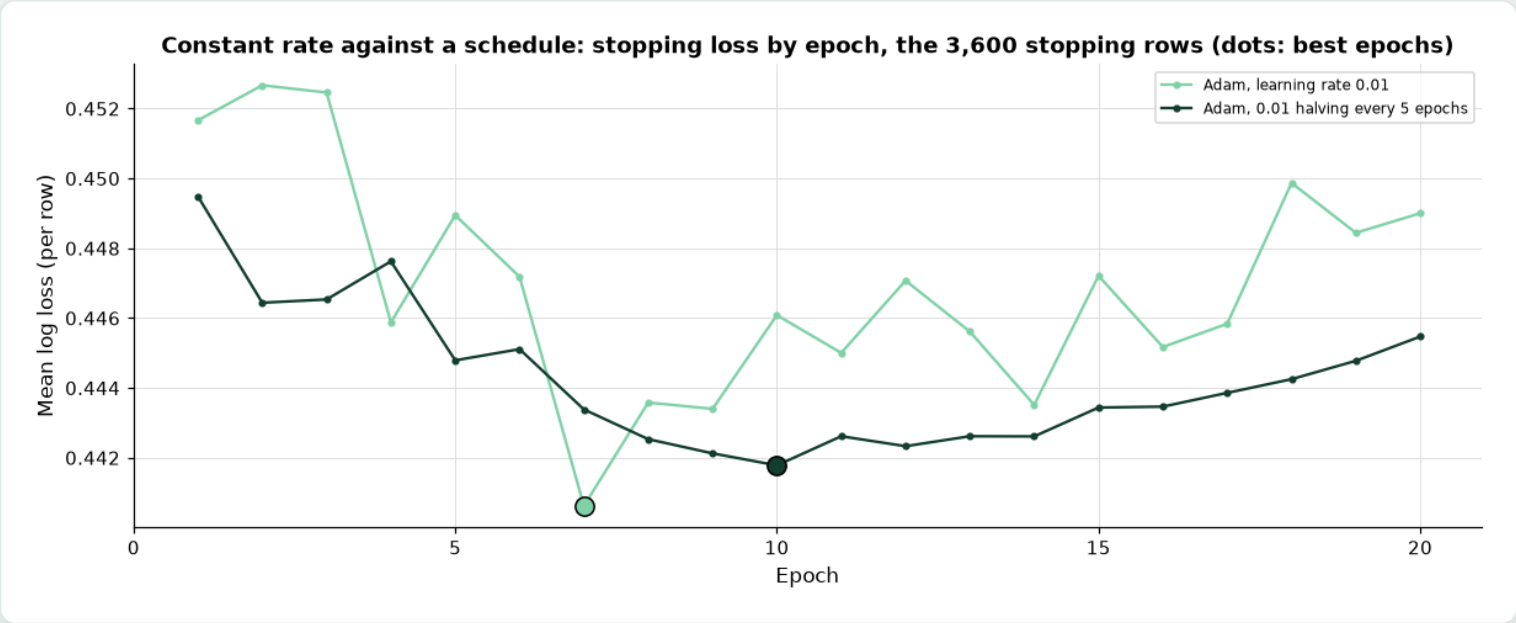
The rate falls a little at every step, and passes Keras's default 0.001 only late in the run.

Five schedule shapes



Pick the shape whose end suits the epochs you will run; ReduceLROnPlateau needs no shape.

A constant rate against the schedule

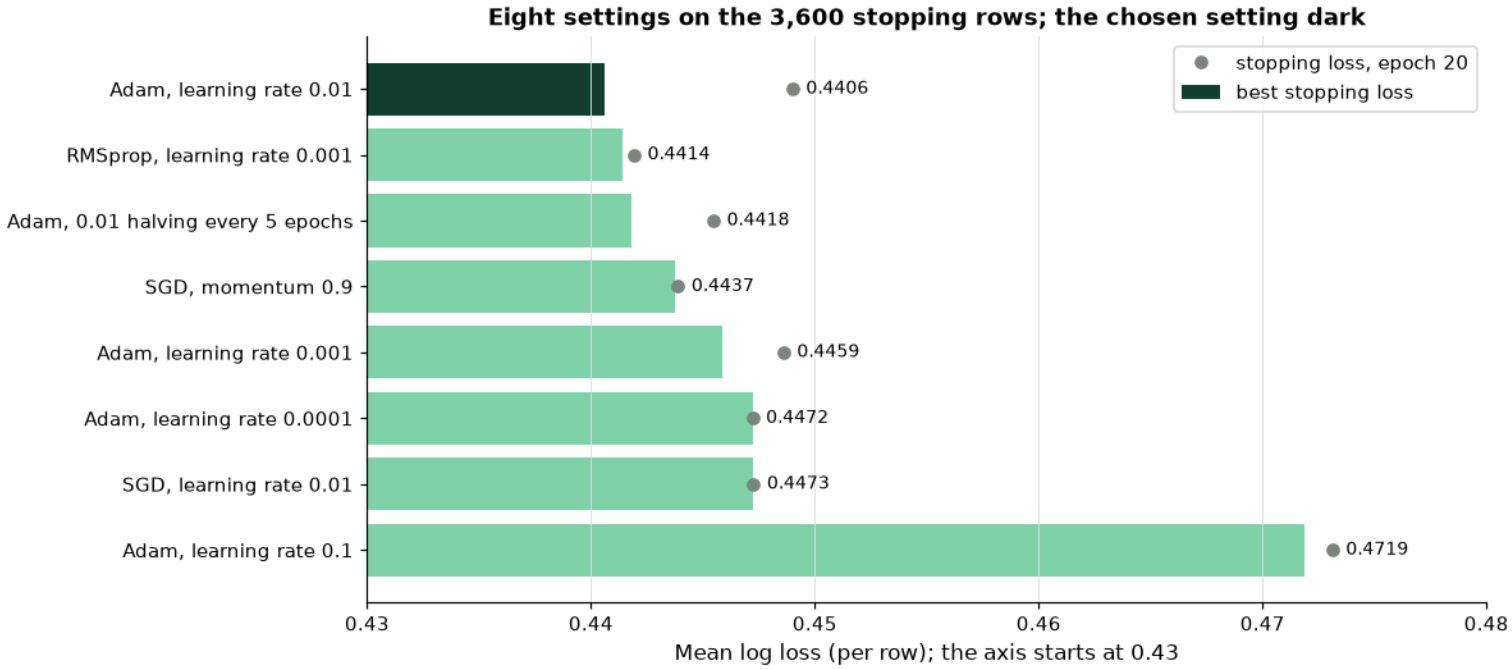


Run	Constant	Schedule
Best stopping loss	0.4406	0.4418
Best epoch	7	10
Stopping, epoch 20	0.4490	0.4455
Largest rise	0.0040	0.0011

The schedule ends lower at epoch 20 and is calmer; it never gets as low as the constant rate's best epoch.

Calmer later epochs; no better fit on this network. ReduceLROnPlateau is the other common choice.

Eight settings, one rule



Lowest stopping loss wins

Adam, learning rate 0.01: 0.4406 at epoch 7. Next: RMSprop, learning rate 0.001, 0.4414. Adam at 0.1 sits far above the other seven.

The axis starts at 0.43. The seven close settings are read against day 4's seed spread.

The choice, then the validation rows once

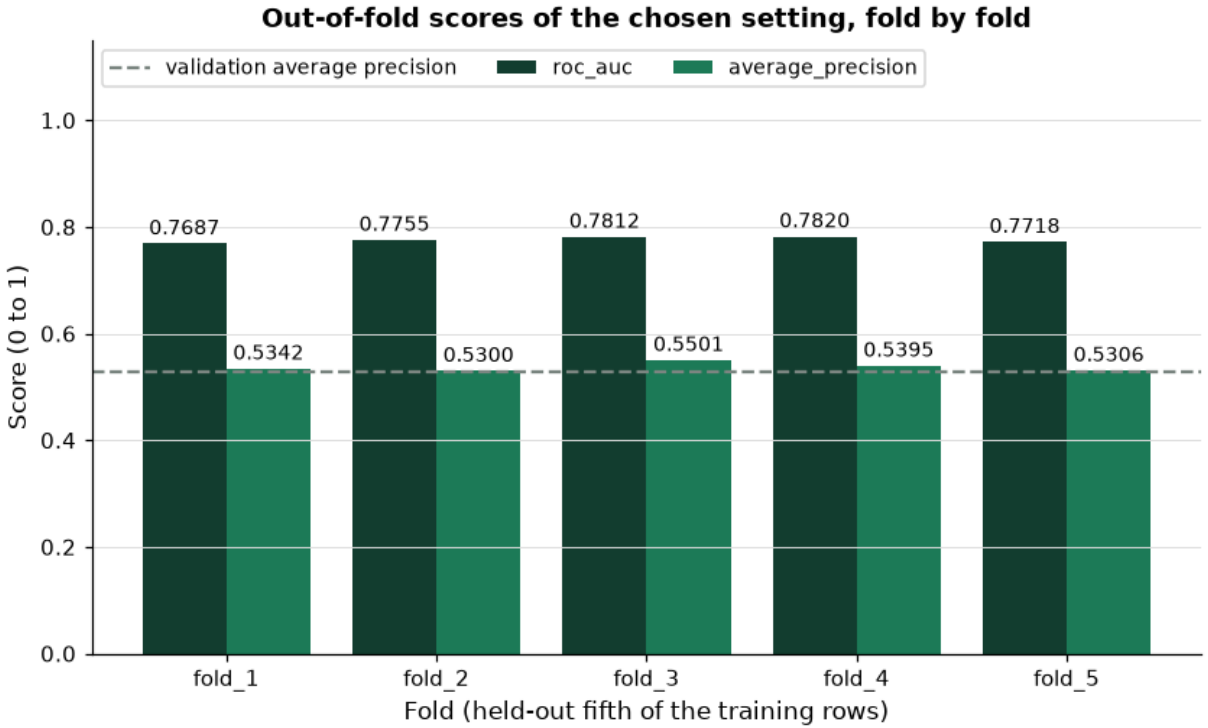
Lowest stopping loss of 8 settings: Adam, learning rate 0.01, epoch 7

Validation rows, once	Best epoch	ROC AUC	Average precision
Adam 0.01 (chosen)	7	0.7745	0.5291
Adam 0.001 (default network)	8	0.7689	0.5258

cv_network, 5 folds: average precision 0.5369 (sd 0.0074); threshold for recall 0.70 0.1867

Small gaps: day 5's bootstrap intervals are the tool. Today reports the numbers and claims nothing beyond them.

The folds, and the validation rows

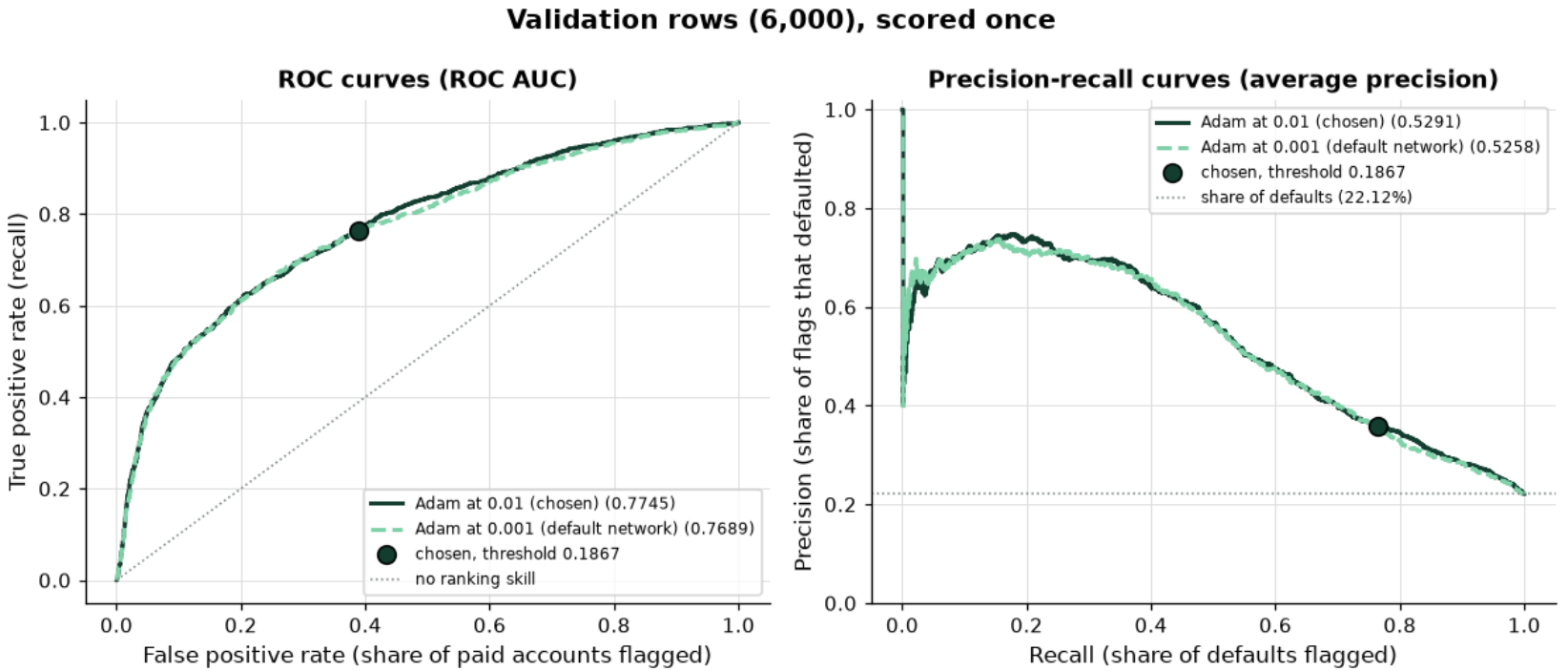


Fold	ROC AUC	Avg. precision
1	0.7687	0.5342
2	0.7755	0.5300
3	0.7812	0.5501
4	0.7820	0.5395
5	0.7718	0.5306

Validation average precision 0.5291: just below the lowest fold. Threshold for recall 0.70: 0.1867.

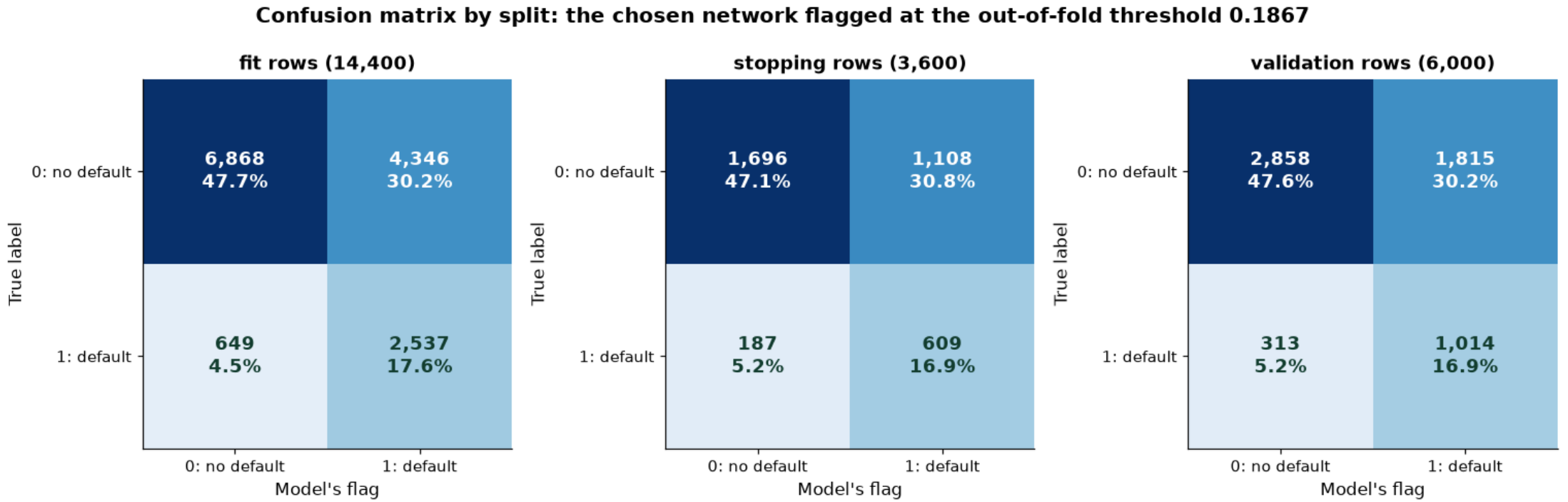
Five folds give a range to read one validation number against.

ROC and precision-recall, validation rows



Close curves, as the scores said. At 0.1867: recall 0.7641, precision 0.3584 on the validation rows.

The chosen network's confusion matrices



Recall at the threshold: fit 0.7963, stopping 0.7651, validation 0.7641.

Excel side by side

Excel side by side (tested in Excel)

`=$K$2*G2-$K$1*E2 then =B2+G3`	momentum, 100 rows: weight 0.771070
`=G2+(E2-G2)*(1-$N$2)`	Adam's m
`=I2+(E2^2-I2)*(1-$N$3)`	Adam's v
`=$N$1*SQRT(1-$N$3^A3)/(1-$N$2^A3)`	alpha 0.015811 at step 1
`=B2-K3*G3/(SQRT(I3)+$N$4)`	Adam, 3 rows: weight 0.646084
`=$B$1*$B$2^(A6/$B$3)`	0.000625 at step 4,500

No optimizer and no network training in Excel. Every number equals Python's within 1e-12 (foundation: =$B$1*C2-$B$2*D2 on hand values too).

Save the day: git stash an experiment

1. Commit

Four modules, today's two functions

2. Log

results.csv and optimizers.csv

3. Stash

push -m: the experiment off the folder

4. Drop

show -p to compare, then drop

```
model,rows,roc_auc,average_precision,threshold,accuracy,precision,recall,f1
adam_0_01_64_32,validation,0.7745,0.5291,0.1867,0.6453,0.3584,0.7641,0.4880
```

The experiment: SGD without momentum at 0.1. Shelved, compared, dropped; git stash pop would bring it back.

The AI check

The docstring is the prompt. The code that comes back runs, and its number looks fine.

1

Docstring

Which rows fit, which rows stop, which rows report: what you asked for.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Assert what the docstring promises. Run it: it fails.

→

4

Fix

Find the rule it broke, fix it, run the test again.

Today: half the bias correction is missing: the first step moves each weight 0.0050 instead of 0.05.

Code that runs is not the same as code that is right.

What you can do now

- Check an optimizer against Keras with apply
- Take momentum steps; say why momentum overshoots
- Take Adam steps; say what the bias correction does
- Choose optimizer and learning rate on stopping rows
- Shelve an experiment with git stash

Week 2, Day 3 Initialization and batch normalization